

REDES DE COMUNICACIONES: DISEÑO INICIAL DEL PROYECTO DE NANOFILES

Ibrahim Cherif Barry

Grupo 1

Subgrupo 1.1

Arturo Trinidad Hoyos

Grupo 1

Subgrupo 1.2

Profesor: Rubén Titos Gil



ÍNDICE

1.	Introducción	-----	3
2.	Formato de los mensajes del protocolo de comunicación con el Directorio	-----	3
3.	Formato de los mensajes del protocolo de transferencia de ficheros	-----	6
4.	Autómatas de protocolo	-----	8
5.	Ejemplos de intercambio de mensajes	-----	9

1. Introducción.

En este documento se especifica el diseño de los protocolos de comunicación UDP y TCP de la práctica de NanoFiles.

La mejora que hemos considerado a la hora de diseñar el protocolo es:

- Comando *serve* con puerto efímero: Ampliar comando *serve* para que el servidor de ficheros pueda utilizar cualquier puerto disponible (ephemeral port) para escuchar, en lugar de solamente el puerto 10000/TCP.

2. Formato de los mensajes del protocolo de comunicación con el Directorio

Para definir el protocolo de comunicación con el *Directorio*, vamos a utilizar mensajes textuales con formato “campo:valor”. El valor que tome el campo “operation” (código de operación) indicará el tipo de mensaje y por tanto su formato (qué campos vienen a continuación).

Tipos y descripción de los mensajes

Mensaje: **ping simple**

Sentido de la comunicación: Cliente → Directorio

Descripción: Permite verificar la conectividad entre el cliente y el Directorio.

Ejemplo:

```
operation:ping\n
\n
```

Mensaje: **pingok**

Sentido de la comunicación: Directorio → Cliente

Descripción: El Directorio está activo

Ejemplo:

```
operation:ping_ok\n
\n
```

Mensaje: **ping con identificador de protocolo**

Sentido de la comunicación: Cliente → Directorio

Descripción: Permite verificar la conectividad entre el cliente y el Directorio. Incluye el campo “protocol” con un identificador

Ejemplo:

```
operation:ping\n
protocol:BALATRO1.1\n
\n
```

Mensaje: directorio accesible

Sentido de la comunicación: Directorio → Cliente

Descripción: La verificación comprueba que el directorio está activo y el protocol_id es correcto.

Ejemplo:

```
operation:ping_welcome\n\n
```

Mensaje: directorio inaccesible

Sentido de la comunicación: Directorio → Clientr

Descripción: La verificación comprueba que el directorio está inactivo, o que el protocol_id es incorrecto.

Ejemplo:

```
operation:ping_denied\n\n
```

Mensaje: lanzar servidor de ficheros

Sentido de la comunicación: Cliente → Directorio

Descripción: Permite al cliente actual como servidor a la vez, lanzando un servidor de ficheros que escuche conexiones en cualquier puerto.

Ejemplo:

```
operation:serve\n\n
```

Mensaje: lanzar servidor de ficheros

Sentido de la comunicación: Cliente → Directorio

Descripción: Permite al cliente actual como servidor a la vez, lanzando un servidor de ficheros que escuche conexiones en cualquier puerto, bajo el protocolo de transporte TCP.

Ejemplo:

```
operation:serve_ok\nport:10000/TCP\n\n
```

Mensaje: solicitud de lista de archivos

Sentido de la comunicación: Cliente → Directorio

Descripción: El cliente solicita al Directorio la lista de archivos disponibles.

Ejemplo:

```
operation:filelist\n\n
```

Mensaje: lista de archivos ofrecida

Sentido de la comunicación: Directorio → Cliente

Descripción: El directorio ofrece la lista de ficheros disponibles al cliente que la solicitó.

Ejemplo:

```
operation:filelist_ok\nfile1:file1.txt,size1,hash1\nfile2:file2.txt,size2,hash2\nfile3:file3.txt,size3,hash3\n\n
```

Mensaje: solicitud de salida del programa

Sentido de la comunicación: Cliente → Directorio

Descripción: Permite al cliente solicitar la salida del programa al directorio.

Ejemplo:

```
operation:quit\n\n
```

Mensaje: salida del programa aceptada

Sentido de la comunicación: Directorio → Cliente

Descripción: El directorio acepta que el cliente salga del programa.

Ejemplo:

```
operation:quit_ok\n\n
```

3. Formato de los mensajes del protocolo de transferencia de ficheros

Para definir el protocolo de comunicación con un servidor de ficheros, vamos a utilizar mensajes binarios multiformato. El valor que tome el campo “opcode” (código de operación) indicará el tipo de mensaje y por tanto cuál es su formato, es decir, qué campos vienen a continuación.

Tipos y descripción de los mensajes

Mensaje: **InvalidCode (opcode = 0)**

Sentido de la comunicación: Cualquiera → Cualquiera

Descripción: Este mensaje indica que se ha recibido un código de operación inválido o desconocido. Se puede utilizar para manejar errores en la comunicación cuando un peer recibe un mensaje con un opcode no reconocido.

Ejemplo:

Opcode (1 byte)
0

Mensaje: **FileNotFound (opcode = 1)**

Sentido de la comunicación: Servidor de ficheros → Cliente

Descripción: Este mensaje lo envía el par servidor de ficheros al par cliente (receptor) de fichero para indicar que no es posible encontrar el fichero con la información proporcionada en el mensaje de petición de descarga.

Ejemplo:

Opcode (1 byte)
1

Mensaje: **GetChunk (opcode = 2)**

Sentido de la comunicación: Cliente → Servidor de ficheros

Descripción: Este mensaje lo envía el cliente para solicitar un fragmento (chunk) de un fichero. Contiene el desplazamiento (offset) desde el inicio del fichero y la cantidad de bytes a recibir.

Ejemplo:

Opcode (1 byte)	File Offset (8 bytes)	Chunk Size (4 bytes)
2	0x00 0x00 0x01 0x00 0x00 0x00 0x00 0x00	0x00 0x02 0x00 0x00

Mensaje: SendChunk (opcode = 3)

Sentido de la comunicación: Servidor de ficheros → Cliente

Descripción: Este mensaje lo envía el servidor de ficheros al cliente en respuesta a un mensaje GetChunk. Contiene el fragmento (chunk) del fichero solicitado.

Ejemplo:

Opcode (1 byte)	File Offset (8 bytes)	Chunk Size (4 bytes)	Chunk Data (n bytes)
3	0x00 0x00 0x01 0x00 0x00 0x00 0x00 0x00	0x00 0x02 0x00 0x00	datos del chunk

Mensaje: UploadFile (opcode = 4)

Sentido de la comunicación: Cliente → Servidor de ficheros

Descripción: Este mensaje lo envía el cliente para solicitar la subida de un fichero. Contiene el nombre del fichero a subir.

Ejemplo:

Opcode (1 byte)	File Name Length (2 bytes)	File Name (n bytes)
4	0x00 0x10	"nanoFilesP2P.zip"

Mensaje: UploadAck (opcode = 5)

Sentido de la comunicación: Servidor de ficheros → Cliente

Descripción: Este mensaje lo envía el servidor de ficheros al cliente para confirmar que la subida del fichero ha sido aceptada correctamente.

Ejemplo:

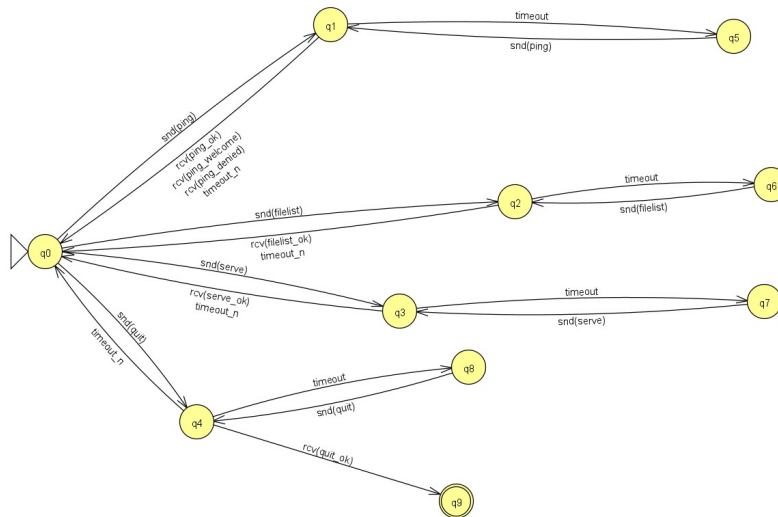
Opcode (1 byte)
5

4. Autómatas de protocolo

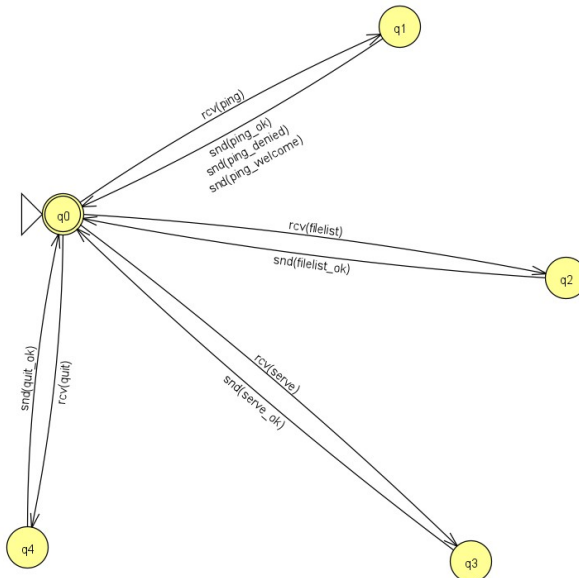
Con respecto a los autómatas, hemos considerado las siguientes restricciones:

- Un cliente no tendrá ping exitoso si no tiene el protocol_id requerido.

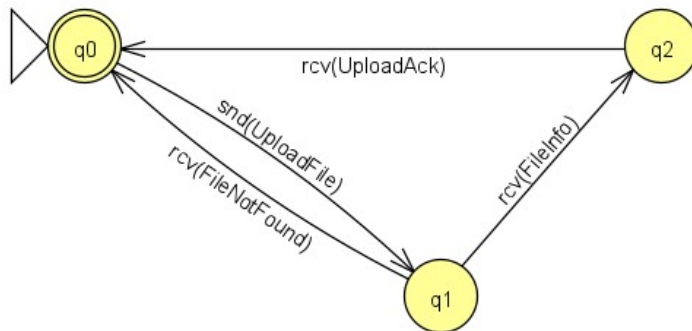
Autómata rol cliente de directorio



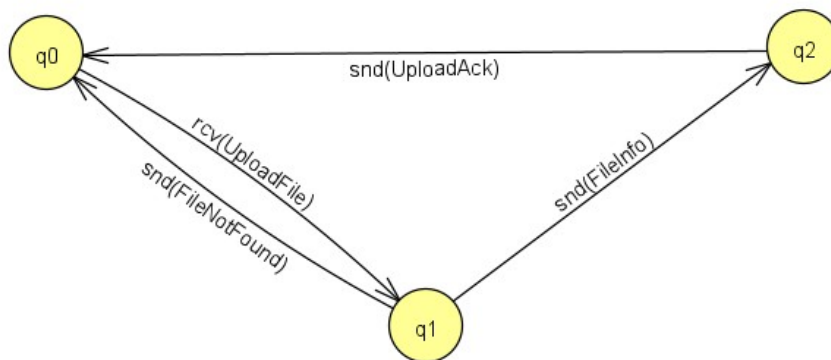
Autómata rol servidor de directorio



Autómata rol cliente de ficheros



Autómata rol servidor de ficheros



5. Ejemplos de intercambio de mensajes

He aquí ejemplos de “conversaciones” ficticias (con valores inventados) haciendo uso de los mensajes definidos en las secciones anteriores y comentando cómo el autómata restringe qué mensaje(s) puede enviar recibir cada extremo de la comunicación en cada instante de la conversación (estado del autómata).

CLIENTE: hace ping al directorio
 operation:ping\n
 protocol:password\n
 \n

DIRECTORIO: está inactivo, o protocol_id incorrecto
 operation:ping_denied\n
 \n

CLIENTE: hace ping al directorio
 operation:ping\n
 protocol:BALATRO1.1\n
 \n

DIRECTORIO: está activo

operation:ping_welcome\n
\n

CLIENTE: pide la lista de ficheros disponibles

operation:filelist\n
\n

DIRECTORIO: Responde especificando la lista de ficheros disponibles

operation:filelist_ok\n
file1:file1.txt,size1,hash1
file2:file2.pdf,size2,hash2
\n

CLIENTE: solicita lanzar servidor de ficheros

operation:serve\n
\n

DIRECTORIO: acepta que el cliente lance servidor de ficheros

operation:serve_ok\n
port:80/TCP\n
\n

CLIENTE: Solicita descargar el fichero *nombrefichero.pdf*

operation:download\n
filename_substring:nombrefichero.pdf\n
local_filename:nuevonombrefichero.pdf\n
\n

DIRECTORIO: Responde aceptando la descarga, especificando el hash además.

operation:file\n
hash:hashhhh\n
\n

CLIENTE: Solicita descargar el fichero *ficheronombre*

operation:download\n

```
filename_substring:ficheronombre\n
local_filename:nombrenuevo.pdf\n
\n
```

```
DIRECTORIO: Responde indicando que no encuentra el fichero
operation:FileNotFound\n
\n
```

```
CLIENTE: Anuncia que quiere salir del programa.
operation:quit\n
\n
```

```
DIRECTORIO: Salida del programa del cliente la acepta.
operation:quit_ok\n
\n
```